Installing MongoDB on Windows

Step 1: Download MongoDB Community Server

1. Open mongodb.com/try/download/community
2. Select Windows as the platform and MSI as the package.
3. Click Download

Step 2: Install the server

1. Run the downloaded .msi file and click Next.
2. Accept the license agreement.
3. Choose the Complete setup type.
4. Keep **"**Install MongoDB as a Service" ticked, so the server starts automatically with Windows.
5. Keep **"**Install MongoDB Compass**"** ticked.
6. Click Install**,** then Finish**.**

Step 3: Install MongoDB Shell (mongosh)

1. Open mongodb.com/try/download/shell
2. Select Windows and the MSI package, then download it.
3. Run the installer and follow the prompts.

Step 4: Verify the installation

1. Close any open Command Prompt windows, then open a new one (press the Windows key, type cmd, press Enter).
2. Type mongosh –version
3. Type mongosh
4. The prompt changes to test>, which means you are connected. Then type: db.version()
5. To leave the shell type exit.

Database Commands

Command: show dbs

Purpose: Lists out databases that exist

```
test> show dbs
admin   40.00 KiB
config  36.00 KiB
local   64.00 KiB
```

Command: use CollectionName

Purpose: Switches to the given database

```
test> use students
switched to db students
```

Command: db

Purpose: Displays the current working database

```
students> db
students
```

Command: help

Purpose: Lists out all commands with its usage.

```
                                        'show logs': Print all logger names.
   exit                                 Quit the MongoDB shell with exit/exit()/.exit
   quit                                 Quit the MongoDB shell with quit/quit()
   Mongo                                Create a new connection and return the Mongo object. Usage: new Mongo(URI, options [op
tional])
   connect                              Create a new connection and return the Database object. Usage: connect(URI, username [
optional], password [optional])
   it                                   result of the last line evaluated; use to further iterate
```

Category: Create (CRUD)

Command: db.CollectionName.insertOne({})

Purpose: Inserts one document at a time.

```
students> db.students.insertOne({name: "Ajay",
|     age: 22,
|     department: "Computer Science",
|     city: "Chennai",
|     skills: ["Python", "SQL"],
|     marks: 85,
|     enrolled: true
|   })
{
  acknowledged: true,
  insertedId: ObjectId('6ad9239d6589142781913518')
}
```

Command: db.CollectionName.insertMany([doc1, doc2])

Purpose: Inserts multiple document simultaneously.

```
students> db.students.insertMany([{
|     name: "Praveen",
|     age: 24,
|     department: "Electronics",
|     city: "Pune",
|     skills: ["Python", "MongoDB"],
|     marks: 76,
|     enrolled: false
|   },
|   {
|     name: "Priya",
|     age: 21,
|     department: "Computer Science",
|     city: "Chennai",
|     skills: ["SQL", "Java"],
|     marks: 89,
|     enrolled: true
|   }])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ad925dc6589142781913519'),
    '1': ObjectId('6ad925dc658914278191351a')
  }
}
```

Category: Read (CRUD)

Command: **db.CollectionName.find({department: 'Computer Science'})**

Purpose: Retrieves top 20 documents.

```
students> db.students.find({department: 'Computer Science'})
[
  {
    _id: ObjectId('6ad9239d6589142781913518'),
    name: 'Ajay',
    age: 22,
    department: 'Computer Science',
    city: 'Chennai',
    skills: [ 'Python', 'SQL' ],
    marks: 85,
    enrolled: true
  },
  {
    _id: ObjectId('6ad925dc658914278191351a'),
    name: 'Priya',
    age: 21,
    department: 'Computer Science',
    city: 'Chennai',
    skills: [ 'SQL', 'Java' ],
    marks: 89,
    enrolled: true
  },
  {
    _id: ObjectId('6ad9276b658914278191351b'),
    name: 'Arun',
    age: 21,
    department: 'Computer Science',
    city: 'Pune',
    skills: [ 'MongoDB', 'Java' ],
    marks: 81,
```

Command: **db.CollectionName.find({condition})**
Purpose: Finds document with the given condition.

```
students> db.students.find({department: 'Computer Science', city: 'Chennai'})
[
  {
    _id: ObjectId('6ad9239d6589142781913518'),
    name: 'Ajay',
    age: 22,
    department: 'Computer Science',
    city: 'Chennai',
    skills: [ 'Python', 'SQL' ],
    marks: 85,
    enrolled: true
  },
  {
    _id: ObjectId('6ad925dc658914278191351a'),
    name: 'Priya',
    age: 21,
    department: 'Computer Science',
    city: 'Chennai',
    skills: [ 'SQL', 'Java' ],
    marks: 89,
    enrolled: true
  }
]
```

Command: db.CollectionName.find({…}, {field1: " ", field2: " "})

Purpose: Finds documents with given condition and displays only required columns.

```
students> db.students.find({department:"Computer Science"},{name:1, marks: 1})
[
  { _id: ObjectId('6ad9239d6589142781913518'), name: 'Ajay', marks: 85 },
  { _id: ObjectId('6ad925dc658914278191351a'), name: 'Priya', marks: 89 },
  { _id: ObjectId('6ad9276b658914278191351b'), name: 'Arun', marks: 81 }
]
```

Command: **db.CollectionName.find().count()**

Purpose: Displays total count

```
students> db.students.find().count()
5
```

Command: **db.CollectionName.find({condition}).count()**

Purpose: Displays the count of total number of documents with department as computer science

```
students> db.students.find({department:"Computer Science"}).count()
3
```

Command: **db.CollectionName.find().limit(no_of_documents)**

Purpose: Displays the given amount of documents.

```
students> db.students.find().limit(2)
[
  {
    _id: ObjectId('6ad9239d6589142781913518'),
    name: 'Ajay',
    age: 22,
    department: 'Computer Science',
    city: 'Chennai',
    skills: [ 'Python', 'SQL' ],
    marks: 85,
    enrolled: true
  },
  {
    _id: ObjectId('6ad925dc6589142781913519'),
    name: 'Praveen',
    age: 24,
    department: 'Electronics',
    city: 'Pune',
    skills: [ 'Python', 'MongoDB' ],
    marks: 76,
    enrolled: false
  }
]
```

Query Operators — Comparison

Command: **db.CollectionName.find({ field: { $gt: value }})**

Purpose: Displays the documents that fall under the condition (i.e, Displays the documents that are aged greater than 22)

```
students> db.students.find({age: {$gt:22}})
[
  {
    _id: ObjectId('6ad925dc6589142781913519'),
    name: 'Praveen',
    age: 24,
    department: 'Electronics',
    city: 'Pune',
    skills: [ 'Python', 'MongoDB' ],
    marks: 76,
    enrolled: false
  },
  {
    _id: ObjectId('6ad9276b658914278191351c'),
    name: 'Nisha',
    age: 25,
    department: 'Electronics',
    city: 'Chennai',
    skills: [ 'Python', 'Excel' ],
    marks: 91,
    enrolled: true
  }
]
```

Similarly, the same structure of command can be used for lesser than **$lt.**

$lte – Lesser than or equal to –> Displays documents that are lesser than or equal to given condition.
$gte – Greater than or equal to –> Displays documents that are greater than or equal to given condition.

Command: **db.CollectionName.find({$or:[{condition1}, {conditon2}]}, {field1:1, field2:1})**

Purpose: $or returns documents when at least one condition matches.

```
students> db.students.find({$or: [{skills: 'Python'}, {skills: 'Java'}]}, {name:1, skills:1})
[
  {
    _id: ObjectId('6ad9239d6589142781913518'),
    name: 'Ajay',
    skills: [ 'Python', 'SQL' ]
  },
  {
    _id: ObjectId('6ad925dc6589142781913519'),
    name: 'Praveen',
    skills: [ 'Python', 'MongoDB' ]
  },
  {
    _id: ObjectId('6ad925dc658914278191351a'),
    name: 'Priya',
    skills: [ 'SQL', 'Java' ]
  },
  {
    _id: ObjectId('6ad9276b658914278191351b'),
    name: 'Arun',
    skills: [ 'MongoDB', 'Java' ]
  },
  {
    _id: ObjectId('6ad9276b658914278191351c'),
    name: 'Nisha',
    skills: [ 'Python', 'Excel' ]
  }
]
```

Command: db.CollectionName.find({field: {$in:[value1, value2]}}, {field1:1, field2:1})
Purpose: $in matches a field against any value in the specified array**.**

```
students> db.students.find({city: {$in:['Chennai', 'Pune']}}, {name:1, city:1})
[
  {
    _id: ObjectId('6ad9239d6589142781913518'),
    name: 'Ajay',
    city: 'Chennai'
  },
  {
    _id: ObjectId('6ad925dc6589142781913519'),
    name: 'Praveen',
    city: 'Pune'
  },
  {
    _id: ObjectId('6ad925dc658914278191351a'),
    name: 'Priya',
    city: 'Chennai'
  },
  {
    _id: ObjectId('6ad9276b658914278191351b'),
    name: 'Arun',
    city: 'Pune'
  },
  {
    _id: ObjectId('6ad9276b658914278191351c'),
    name: 'Nisha',
    city: 'Chennai'
  }
]
```

Command: db.CollectionName.find({field: {$nin:[value1, value2]}}, {field1:1, field2:1})

Purpose: $**nin** matches documents where the specified field does not exist.

```
students> db.students.find({city: {$nin:['Chennai', 'Pune']}}, {name:1, city:1})
```

Category: Update (CRUD)

Command: **db.CollectionName.updateOne({filter}, {$set: {field:newValue}})**

Purpose:updateOne() updates at most one matching document. (i,e In this example the **Command** matches with the given objectId and updates its department to “Electronics”)

```
students> db.students.updateOne({_id: ObjectId('6ad96205a1c3856ddafe5d01')}, {$set: {department: 'Electronics'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Command: **db.CollectionName.updateMany({filter}, {$set: {field:newValue}})**

Purpose: updateMany() updates all documents that match the filter (i,e, In this example the command matches with the electronics department and updates it as “Electronics and communication”)

```
students> db.students.updateMany({department: "Electronics"}, {$set: {department: "Electronics and Communication"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

Category: Delete (CRUD)

Command: db.CollectionName.deleteOne({unique_id})

Purpose: deleteOne matches the unique objectId given and deletes that one document.

```
ts> db.students.deleteOne({_id: ObjectId('6ad9276b6589142781913 51c')})
{ acknowledged: true, deletedCount: 1 }
```

Command: db.CollectionName.deleteMany({filter})

Purpose: deleteMany() deletes every document that matches the given filter.

```
students> db.students.deleteMany({marks: {$lt:90}})
{ acknowledged: true, deletedCount: 4 }
```